Extracting Inline Tests from Unit Tests

Anonymous Author(s)

ABSTRACT

Inline tests were recently proposed for validating individual program statements; they allow developers to provide test inputs, expected outputs, and test oracles immediately after a target statement. But, existing software can have many target statements. So, automatic generation of inline tests is an important next step towards increasing their adoption. We propose ExLi, the first technique for automatically generating inline tests. ExLi extracts inline tests from unit tests; it first records all variable values at a target statement during execution of unit test. Then, ExLi uses those values as test inputs and test oracles in an initial set of generated inline tests. Target statements that are executed many times could have redundant initial inline tests. So, ExLi performs two rounds of reduction. First, ExLi uses a novel, lightweight, and on-the-fly context-aware coverage-based approach to reduce the initial set. Then, ExLi uses test-suite minimization based on mutation analysis to perform further reduction. We implement ExLi for Java and use it to generate inline tests for 642 target statements in 31 open-source programs. ExLi reduces 11091 initially generated inline tests by 14.9x without loss in mutation scores. The final set of generated inline tests kills 40.6% and 286.4% more mutants, respectively, than developer-written and Randoop-generated unit tests. That is, ExLi generates inline tests that can improve the fault-detection capability of the test suites from which they are extracted.

1 INTRODUCTION

Inline tests [29] enable developers to test individual program statements, thereby increasing the fault-detection capability of test suites. Inline tests are complementary to existing levels of test granularity—unit tests, integration tests, and end-to-end tests. Inline tests can help find single-statement bugs, which often occur [24, 25] but elude unit tests [28]. Inline tests can also provide other software engineering benefits, e.g., they document complex target statements and they can be easier to co-evolve with code than unit tests.

Two tools have been developed to provide framework-level support for inline testing. These tools make it easier for developers to write inline tests and they increase the chances for the relatively new inline testing paradigm to be more broadly adopted. One tool supports inline testing in Python [30, 47] and it is integrated with `pytest`, the most popular testing framework for Python [48]. A tool for Java was presented in the original inline testing paper [29].

Automatic generation of inline tests is an important next step towards increasing their adoption for two reasons. First, automatic generation can reduce the manual developer effort required to retrofit inline tests into existing code bases that have many target statements. Second, automatic generation can enable future inline testing research, which will likely require evaluation on more inline tests than exist today. For example, Liu et al. [29] had to simulate runtime costs by repeating the execution of their 152 manually written inline tests thousands of times.

We propose ExLi, the first technique for automatically generating inline tests. ExLi extracts inline tests from unit tests. Unit tests are an attractive source of inline tests since they are abundant in practice or they can be automatically generated [13, 42]. In turn, the extracted inline tests can help find single-statement bugs that unit tests miss [28]. Extracted inline tests can also help find bugs that occur in executed statements that are deeply-nested in conditional expressions. Prior work showed that automatically generated unit tests can miss such deeply nested bugs [2]. Since ExLi is a first step towards inline test generation, we assume that unit tests correctly exercise the code under test.

Given the code under test, a target statement, and unit tests that cover the target statement, ExLi generates a set of inline tests for the target statement. ExLi can automatically discover the four kinds of target statements that Liu et al. [29] used, and select unit tests that cover them. ExLi is agnostic to the source of unit tests; they can be manually written or automatically generated. ExLi outputs a new version of the code under test in which the target statement is immediately followed by generated inline tests.

ExLi first instruments the code under test to record all observed variable values in the target statement during unit testing. Then, the recorded values are used to automatically generate inline tests. For example, consider target assignment statements, recorded values of right-hand side variables are used as input values in generated inline tests and left-hand side variable values, corresponding to computation results, are used as expected values. Our ExLi prototype also supports the generation of inline tests for declaration statements and expressions in `if` conditions; support for more locations of target statements can be added in the future.

Inline tests are co-located with target statements, so an important concern is that readability could be degraded if too many inline tests are generated per target statement. Compilation could also fail if adding the generated inline tests causes a method's body to exceed the maximum allowable size [41]. Too many inline tests can be generated for target statements in which many sets of variable values are observed during unit testing. Such many-valued target statements could be covered by many unit tests, or they may be in loops. In a particularly egregious case, 88682 sets of variable values were recorded for a target statement during our experiments.

To address the concern of generating too many initial inline tests per target statement, ExLi uses a two-round redundancy reduction process. The first round uses a novel, lightweight, and on-the-fly (i.e., during unit test execution) context-aware coverage-based approach to remove inline tests that do not cover the target statement or its context differently than prior ones. The first round is broadly applicable to all covered target statements. The first round is also lightweight; it analyzes coverage information of each inline test on the fly, as soon as values are extracted. Saving all recorded values together with their coverage information for after-the-fact analysis would be more costly.

The first round's coverage-based reduction could produce inline tests that are redundant with respect to fault-detection capability, as measured by mutation analysis [22, 50]. So, second round performs mutation analysis on the target statement and uses the

results to further reduce the set of generated inline tests. If the first-round reduced set of inline tests has a lower mutation score than the initial set, ExLi adds inline tests that kill surviving mutants from the initial set to the first-round reduced set. Once parity of mutation scores is achieved, test-suite minimization is applied to remove inline tests that are redundant with respect to the set of killed mutants. Both rounds of reduction in ExLi are essential for efficiently and effectively reducing the number of inline tests per statement without losing fault-detection capability.

We implement ExLi for Java and apply it to 642 target statements in 31 open-source programs. ExLi generates an initial set of 11091 inline tests. ExLi's first round of reduction on the initial set produces 1032 inline tests. In the second round, ExLi uses the universalmutator [17], a mutant generation tool, to mutate target statements and test suite minimization [6, 7, 20] to perform further reduction, yielding a final set of 743 inline tests. So, ExLi reduces the initial set of generated inline tests by 14.9x.

We further evaluate whether generated inline tests enhance the fault detection capability of test suites from which they are extracted. The final set of generated inline tests kills 40.6% more mutants than developer-written unit tests, and 286.4% more mutants than unit tests generated by Randoop [42, 49]. Our manual inspection shows why generated inline tests can provide additional fault-detection capability. The unit tests reach the target statements and infect program state, but those unit tests lack "local" oracles at the target statement. That is, errors induced by the mutants do not propagate to the assertions in the unit tests, or those assertions do not check the relevant parts of program state.

This paper makes the following contributions:

- ⋆ **Technique.** ExLi is the first technique for automatically generating inline tests; it extracts inline tests from unit tests.
- ⋆ **Reduction approach.** ExLi uses a novel inline test reduction approach that is based on how a target statement is covered.
- ⋆ **Evaluation.** ExLi's reduction strategy is effective, yielding inline tests that improve fault-detection capability of unit test suites.
- ⋆ **Dataset.** ExLi generates the largest dataset of inline tests to date. ExLi and our dataset can enable future work on inline tests.

Our dataset is at https://github.com/pp7523338/exli.git.

2 EXAMPLE

Figure 1 shows an example code with a target statement and inline tests that ExLi generates for that target statement after both rounds of reduction. The example is simplified from `logstash-gelf` [31]. Method `setAdditionalFields` splits `spec` using "," as the delimiter, stores the results in the `fields` variable, and adds each `field` in `fields` that contains "=" to `gelfMsg`. Line 8 is the target statement; it finds the index of first occurrence of `EQ` in `field`.

Suppose a developer gives this target statement as input to ExLi, e.g., because it is in a loop and it is used by lots of other methods. Line 9 is an inline test that ExLi generates. All inline tests consist of three parts. First, the "Declare" part—`new Here()`—signifies that the current statement is an inline test. Second, the "Assign" part—`given(field, "profile.requestStart.ms")`)—allows developers to provide inputs to the inline test. Third, the "Assert" part—`checkEq(index, −1)`)—allows developers to specify a test oracle that includes the expected output. In Figure 1, given the input for `field`, the `index` variable that is being computed in the target statement should equal −1 for the inline test on line 9 to pass.

```
1  public static final String MULTI_VALUE_DELIMITTER = ",";
2  public static final char EQ = '=';
3
4  public static void setAdditionalFields(String spec, GelfMsg gelfMsg) {
5   if (null != spec) {
6    String[] properties = spec.split(MULTI_VALUE_DELIMITTER);
7    for (String field : properties) {
8     final int index = field.indexOf(EQ);
9     new Here().given(field, "profile.requestStart.ms").checkEq(index, -1);
10    new Here().given(field, " mdcName='long']").checkEq(index, 8);
11    if (-1 == index) { continue; }
12    ... // add field to gelfMsg
13    }}}
```

Figure 1: A target statement with ExLi-generated inline tests.

The example target statement is executed 2413 times with 215 unique set of variable values during unit testing. But, directly generating 215 inline tests to check one statement could be an overkill for two reasons. First, many of the 215 sets of values are redundant because they exercise the target statement in the same way. So, running them all is wasteful. Second, adding 215 inline tests after this target statement will likely make the code hard for developers to read and maintain. So, there is a need to reduce the number of generated inline tests by eliminating redundancy. ExLi's two-round reduction process reduces those 215 inline tests into the two that are shown in Figure 1 without loss in bug-detection capability.

3 TECHNIQUE

3.1 Overview

Figure 2 shows ExLi's procedure for generating inline tests. The inputs are the code under test, the unit tests, and an optional line number of the target statement (not shown). The output is a new version of the code under test where target statements are followed immediately by generated inline tests.

In step ①, `TargetStmtFinder` parses the abstract syntax tree (AST) of the code under test and extracts the target statement(s). Then, `VariablesFinder` identifies the variables used in each target statement (step ②). Next, `Instrumenter` adds code *before* each target statement to collect the values of input variables and *after* each target statement to collect the output variable value (step ③).

When running the unit tests on the instrumented code, too many sets of input and output variable values could be collected for some target statements. Using too many sets of values may result in generating too many inline tests, which (1) could degrade readability, since inline tests are co-located with code; or (2) cause compilation failure, if adding generated inline tests causes a method to exceed the maximum allowable size [41]. But, we observe in our experiments that many sets of collected variable values are redundant with respect to one another: they exercise the target statement and its context in the same way. (Recall that, from a unit testing point of view, the sets of values that ExLi collects are intermediate values.). We find that the percentage of unique values to all values is 15.3%.

Figure 3 shows how we instrument the example code in Figure 1 to collect the input and output values of the target statements during unit testing. There are three variables in the target statement on

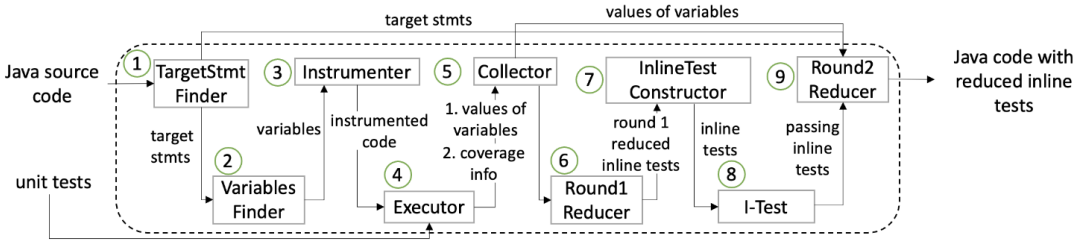

Figure 2: The steps in ExLi's workflow.

```
1  public static void setAdditionalFields(String spec, GelfMsg gelfMsg) {
2   if (null != spec) {
3     String[] properties = spec.split(MULTI_VALUE_DELIMITTER);
4     for (String field : properties) {
5      try {
6      collectCov();  //cov1
7      collectInputs(field, EQ);
8      final int index = field.indexOf(EQ);
9      collectOutputs(index);
10     collectCov();  //cov2
11     if (-1 == index) { continue; }
12     ...
13     } finally {
14      collectCov();  //cov3
15     }
16    } } }
```

Figure 3: Example showing ExLi's instrumentation.

line 8: `index`, `fields`, and `EQ`. The values of the `fields` and `EQ` variables are inputs to the target statement, i.e., right-hand side values, so we collect them before the target statement (line 7). The value of variable `index` is the output, i.e., a left-hand side value, so we collect it after the target statement (line 9).

ExLi uses knowledge about changes to the coverage rate of instructions *around* the target statement, i.e., the context, to reduce the size of the set of collected values. To do so, the `Instrumenter` (step ③) also adds code to collect coverage information about the target statement's context. Specifically, `Instrumenter` adds code at the end of the basic block that contains the target statement to collect changes to the instruction coverage rate. The idea is simple. Two sets of input and output values that exercise the same instructions in the target statement and its context in the same way are likely redundant with respect to one another for inline testing.

The `Executor` (step ④) runs unit tests on the instrumented code, and the `Collector` stores in memory the *unique* sets of variable values and the coverage rates of the target statements (step ⑤). Based on the aforementioned context-aware coverage idea, in step ⑥, the `Round1Reducer` uses the procedure in Algorithm 1 to perform first-round reduction on the set of stored input and output variable values per target statement. Procedure `canSaveInlineTest` accepts, as inputs, three dictionaries of coverage mapping (`c1`, `c2`, and `c3`) and the line number of a target statement (`l0`). It returns a Boolean value indicating if current variable values in the target statement's should be saved for inline test generation.

Figure 3 also shows how we collect the coverage rate of the target statement's context, which ExLi uses as feedback to generate a reduced set of inline tests. To check whether a set of input and output values increase the instruction coverage rate, we collect

Algorithm 1 Round1 Reducer

global variable: LineToCovered: $\{L \rightarrow \{L\}\}$
Inputs: L: line, N: number of covered instructions, cov: $\{L \rightarrow N\}$
Outputs: true or false

1: **procedure** canSaveInlineTest(cov: c1, cov: c2, cov: c3, L: l0)
▷ $l0$ is the target statement
2: stmtCovChanged ⟵ covChanged(c1, c2, l0)
3: contextCovChanged ⟵ covChanged(c2, c3, l0)
return stmtCovChanged ∨ contextCovChanged
4: **procedure** covChanged(cov: c1, cov: c2, L: l0)
5: change ⟵ false
6: **for** $l \in$ c2.keys() **do**
7: **if** $l \notin c1 \vee c1[l] < c2[l]$ **then** ▷ line l covers more
8: **if** $l \notin$ LineToCovered[l0] **then** ▷ line $l0$ not covered by l
9: change ⟵ true
10: LineToCovered[l0] ⟵ LineToCovered[l0] $\cup \{l\}$
return change

(1) the instruction coverage rate just before the target statement (line 6, cov1), (2) the instruction coverage rate right after the target statement (line 10, cov2) and (3) the coverage rate of target statement's context, which is at the end of the block containing the target statement (line 14, cov3).

ΔCov_1 = cov2 \ cov1 shows whether executing the target statement increases the number of covered instructions. ΔCov_1 is not empty when the target statement (1) invokes other methods in the project, (2) invokes other methods in the standard library, or (3) contains a condition or uses a ternary operator—current inputs now cover different instructions in the statement. The coverage rates can range from not covered to partially covered to fully covered.

ΔCov_2 = cov3 \ cov2 shows whether the number of covered instructions in the context of the target statement increases. If ΔCov_1 and ΔCov_2 show that the the current set of input and output values do not cover more instructions than prior sets, ExLi skips the current set. That is, no inline test is generated from the current set of input and output values if they do not cover the target statement or its context in a different way than prior sets.

`InlineTestConstructor` (step ⑦) synthesizes inline tests using the set of input and output values that remain after applying `Round1Reducer` in step ⑥ . To do so, the value collected for each input variable is used in a `given(...)` calls, which can be chained. That is, the inline test being constructed will assign each input value to the corresponding variable when testing the target statement. Then, the matching value of the output variable is used

Table 1: API used to filter statements.

Types	API
Regex	Matcher.matches()|Matcher.find()|Matcher.group()
String	String.split()|String.substring()|String.indexOf()| String.format()|String.replace()
Bit	» | « | & | | | ˆ | ˜ | &= | |= | ˆ= | »= | «=
Stream	Stream.of()|*.stream()

in a `check_eq(...)` construct. That is, the inline test being constructed will check that the observed value that is computed by the target statement from the assigned input values is equal to the output variable value that was recorded during unit testing. `InlineTestConstructor` also edits the code under test to insert the constructed inline tests immediately after the target statement.

In the penultimate step ⑧, ExLi uses I-Test (a prototype inline testing framework for Java) to execute each generated inline test. If any of the executed inline tests fail, ExLi filters it out: the failing inline test is removed from the code under test. Removing failing tests makes subsequent comparison of mutation scores fair and easy to interpret.

`Round2Reducer` is the last step in ExLi (step ⑨). `Round1Reducer` uses a coverage-based approach to reduce the set of recorded input and output values. So, the set of inline tests produced so far may have redundancy with respect to killed mutants. `Round2Reducer` aims to further reduce the initial set of generated inline tests generated by removing those that are redundant with respect to mutants killed on the target statement. `Round2Reducer` first mutates the target statement. Then, `Round2Reducer` checks if the set of inline tests produced by `Round1Reducer` have a reduced mutation score relative to using all input values in the `Collector` (step ⑤) to kill mutants. There are two possible outcomes of this check.

(1) There is no loss in mutation score. In this case, `Round2Reducer` directly applies test-suite minimization to obtain a possibly smaller set of inline tests that is minimal with respect to the set of mutants killed by using all input values in the `Collector`.
(2) There is a loss in mutation score. In this case, there must be an input that kills the surviving mutants in the set that `Collector` tracks, but our coverage-based approach was unsound and it missed to create an inline test for that input. So, `Round2Reducer` employs a feedback loop to generate an inline test that can kill the mutants to restore parity to the mutation scores before performing test-suite minimization.

`Round2Reducer` preserves fault-detection capability because it always performs minimization of the inline tests from `Round1Reducer` against the same set of mutants that would be killed if one generated the set of all possible inline tests (without first-round reduction).

Remark 1. Conceptually, ExLi could directly use traditional test-suite minimization with respect to mutants on the target statement to reduce the sets of values that are collected during unit testing. Instead, we make the design choice to use a coverage-based approach for three reasons. First, using mutants for minimization requires to first generate inline tests for all the collected sets of values. It is not always possible to do so due to limits on method sizes [41]. Second, using a coverage-based approach has the benefit that we can use mutation testing as a sanity check of the fault-detection capability of the reduced set of inline tests. There would be no automated sanity check if mutation testing is used initially. Lastly, ExLi will need to preserve all inline tests for target statements in which no mutant is created. So, if ExLi only uses mutation analysis and a frequently covered target statement has no mutants, then readability may degrade because too many inline tests are generated.

Remark 2. Implicitly, generating inline tests from unit tests induces a trade-off space among the competing goals of good readability, high coverage, and high fault-detection capability. Since inline tests are co-located with the code under test, fewer inline tests will likely lead to better readability, but at the cost of possibly lower coverage or lower fault-detection capability. We design ExLi to have high readability and high fault-detection capability at the cost of possible loss in the instruction coverage of the target statement or its context. Specifically, round two reduction is not guaranteed to preserve the coverage rate achieved by the inline tests from round one reduction. We optimize for code maintenance settings where high readability with high fault-detection capability is likely preferable to poor readability. ExLi can be configured to optimize differently along the trade-off space. Also, now that ExLi can generate many more inline tests than previously possible, future work can more easily perform user studies of developers' trade-off preferences.

4 IMPLEMENTATION

We describe our ExLi implementation, using the same step numbers as in Section 3.1 to make our description easier to follow.

① **Find target statements**. Users can give ExLi the line number of a target statement to generate inline tests for. Alternatively, a user can generate inline tests in a batch by specifying the kind of target statement. To prove our concept, ExLi currently only supports batch generation for the same four kinds of Java target statements as Liu et al. [29]: regular expressions, string manipulation, bit manipulation, and stream processing. Given a kind of target statement, the `TargetStmtFinder` component in ExLi searches for target statements that use APIs that are commonly used in the kinds of statements of interest. Table 1 lists the APIs that ExLi looks for. Liu's et al.'s prototype searches over program text. But, ExLi parses the AST to find target statements, improving search accuracy.

② **Identify variables**. ExLi's `VariableFinder` component parses the AST of a given target statement to identify its variables. `VariableFinder` also tracks variables that are in the scope of the block containing the target statement, to avoid selecting target-statement scoped variables. Consider this target statement:

```
String str = list.stream().map(item -> item.replace("a",
    "b")).collect(Collectors.joining(","));
```

The variable called `item` is only accessible within the target statement, so `VariableFinder` does not collect it.

③ **Instrument the Code**. ExLi generates inline tests from sets of input and output variable values in the target statement. So, the `Instrumenter` component first instruments the code under test so that these values can be collected during unit testing. The `Instrumenter` works based on the syntax of the target statement and ExLi currently supports instrumenting three syntactic locations of target statements (more can be added in the future):

(1) *The target statement is a condition in an* `if` *statement.* Figure 4 shows an example from project `json-schema-validator` [23]. Line 8 is the target statement; it checks if `value` matches a pattern. ExLi adds instrumentation on Line 7 before the `if`

```
1  public String[] match(String value) {
2      ...
3      for (int i = 0; i < patterns.length; i++) {
4          try {
5              Matcher matcher = patterns[i].matcher(value);
6              collectCov();  //cov1
7              collectInputs(matcher);
8              if (matcher.matches()) {
9                  executed(true);
10                 collectCov();  //cov2
11                 int count = matcher.groupCount();
12                 String[] groups = new String[count];
13                 for (int j = 0; j < count; j++) {
14                     groups[j] = matcher.group(j + 1);
15                 }
16                 return groups;
17             }
18             else{
19                 executed(false);
20             }
21         } finally {
22             collectCov();  //cov3
23         }
24     }
25     return null;
26 }
```

Figure 4: Example of a condition with instrumentation.

statement (to collect variables used in the target statement), at the beginning of the `then` branch (to mark the that the target statement is executed, Line 9), and at the beginning of the `else` branch (to mark that the target statement was not executed, Line 19). If there is no `else` branch, ExLi adds an empty `else` block first (Line 18).

(2) *The target statement is a declaration statement.* ExLi adds instrumentation before the target statement to collect values of right-hand side variables and after the target statement to collect the values of the left-hand side variables.

(3) *The target statement is an assignment statement.* ExLi adds instrumentation to collect left- and right-hand side variable values before the target statement and to collect left-hand side variable values after the target statement. Instrumentation of left-hand side values before the target statement is done to avoid overwriting the left-side input value in statements like `a += 1;`.

ExLi's first round reduction uses coverage rates, so it also adds instrumentation to collect instruction coverage rate (1) before the target statement (`cov1`), (2) after the target statement (`cov2`), and (3) at the end of the block containing the target statement (`cov3`).

Handling Special Instrumentation Cases.

(1) ExLi performs special processing for prefix and postfix operators in the indices of array access expressions. For example, if the target statement is `a[i++] = i » 1;`, then ExLi adds instrumentation to obtain the value of `a[i-1]` instead of `a[i++]`.

(2) Some target statements are in `if` blocks that have jump (`return`, `break`, `continue`, `throw`, etc.) instructions in the `then` branch and in the `else` branch. Adding instrumentation at the end of the blocks in those branches will cause compilation errors due to unreachable code. To avoid compilation errors, ExLi wraps the parent node of the target statement in the AST in a `try` block. In a further special case, the target statement's parent could be a constructor body whose first statement is another constructor call (e.g., `super()` or `this()`). ExLi excludes such constructor calls from the wrapping `try` block that it generates.

(3) To measure coverage, we use JaCoCo [38] and set it to instrument Java standard library classes at class loading time. But, since `java.lang.String` is used by JaCoCo before class loading, it does not get instrumented. If `java.lang.String` is not instrumented, ExLi could miss changes in coverage rates, or values in target statements that call `java.lang.String` methods, e.g., `b = a.replace("at", "@");`. To avoid such problems, our implementation uses wrapper classes that we write for `java.lang.String` and `java.util.Matcher` so that the method calls of these classes can be instrumented. It is necessary to wrap `java.util.Matcher` because some `java.lang.String` methods that are used by our evaluation subjects depend on it.

④ Run unit tests and ⑤ collect variable values. The `Executor` component runs unit tests against the instrumented code and the `Collector` component stores the values of input and output variables that are observed during execution. ExLi is agnostic to the source of unit tests; they can be manually written or automatically generated. We currently use Randoop for automatic unit test generation; future work can investigate other test generators. Our ExLi prototype allows users to configure the maximum number of inline tests to generate per target statement. We use the default maximum value of 100. Very few target statements in our evaluation reach this limit regardless of the kind of unit test suite.

When the variable of interest if of a primitive type, a wrapper type for a primitive type, a `String`, an array of a primitive type, or an array of `String`, the `Collector` directly stores the collected values. Otherwise, the `Collector` uses XStream [32] to serialize and deserialize the variable values as needed. The `Collector` also stores the coverage rates of the target statement and its context for use in the first round of reduction.

⑥ First round reduction and ⑦ inline test construction. The `Round1Reducer` component reduces redundancy among collected sets of variable values that cover a target statement in the same way. To do so, it takes in a target statement and the sets of collected variable values for that target statement. Then it runs our implementation of Algorithm 1 to produce a set of variable values that is minimal with respect to the current coverage levels of the target statement. It is this minimal set that the `InlineTestConstructor` component uses to construct inline tests after each target statement. Inline test construction from ASTs is straightforward.

⑧ Running inline tests. The `Executor` is currently I-Test [29], with minor modifications that we make to how inline tests are parsed and run in a separate context.

⑨ Second round reduction. The `Round2Reducer` uses mutation analysis and test-suite minimization to further reduce the generated inline tests. To do so, `Round2Reducer` first uses the Universal Mutation tool [17] to mutate each target statement. Then, all sets of values that uniquely cover that statement (the contents of `Collector`) as well as all inline tests resulting from first round reduction are run to measure their mutation scores.

Round one reduction is coverage based. So, mutants may survive first-round reduced inline tests even if such mutants are killed by the values in the un-reduced set that `Collector` stores. That is, the first-round reduced inline tests could have a lower mutation score. So,

Table 2: Projects statistics. T(s): Time to run tests. #: No. of test methods. Lcov: Line coverage. Bcov: Branch coverage.

PID	Project	SHA	LOC	Unit tests				Randoop tests			
				T(s)	#	Lcov	Bcov	T(s)	#	Lcov	Bcov
P1	Amazon-sqs-java-extended-client-lib	58fed25	1,288	3.26	36	69%	63%	5.93	16,400	18%	7%
P2	Aquarius-sdk-java	8f4edb9	21,634	2.29	165	1%	50%	3.94	7,215	62%	41%
P3	Build-helper-maven-plugin	f1fac8c	2,424	4.24	55	14%	7%	32.18	19,884	31%	23%
P4	Core	4d394a9	10,233	7.62	63	52%	47%	5.66	3,741	40%	23%
P5	Cyclonedx-core-java	d933705	6,011	6.61	371	67%	37%	16.37	3,286	55%	28%
P6	Elasticsearch-analysis-pinyin	01dda56	2,169	3.12	20	78%	76%	3.70	7,341	35%	24%
P7	Email-ext-plugin	699277c	13,190	435.01	334	66%	54%	23.43	7,032	23%	11%
P8	Extclassgenerator	40ad147	6,271	3.42	99	78%	70%	2.58	763	24%	11%
P9	Fabric-sdk-java	da35400	33,677	215.18	430	12%	9%	16.81	8,697	25%	20%
P10	Geoip-api-java	1030316	11,526	2.49	11	22%	5%	4.72	10,869	17%	4%
P11	Gerrit-rest-java-client	a0bf7cc	14,594	3.61	244	51%	35%	7.22	10,961	53%	34%
P12	Java-asana	52fef9b	5,572	1.87	67	24%	79%	3.77	1,476	89%	36%
P13	Jopenfst	14c4a1d	5,180	1.87	88	84%	74%	3.82	7,843	36%	33%
P14	Jproc	b872abf	1,189	14.14	38	89%	89%	5.82	4,867	31%	23%
P15	Jscep	b20e944	6,310	38.31	210	80%	73%	9.28	1,412	32%	29%
P16	Liquibase-oracle	6ab7dea	7,170	3.27	140	37%	9%	4.67	11,098	67%	49%
P17	Logstash-gelf	66debd8	13,130	9.17	269	78%	70%	22.62	11,467	53%	30%
P18	Maven-site-fixer	60244c0	1,689	3.49	73	88%	84%	8.83	2,098	24%	8%
P19	Messageml-utils	b4c75c6	21,765	5.33	1,170	89%	81%	4.54	2,886	44%	27%
P20	Mp3agic	407f7a9	9,907	2.71	495	88%	68%	5.86	10,147	68%	49%
P21	Nfe	1ccdba7	133,698	3.61	3,600	32%	13%	9.49	17,451	49%	14%
P22	Ognl	5c30e1e	18,190	10.85	939	70%	61%	2.38	494	29%	17%
P23	Pdfcompare	1259ef2	4,213	10.25	73	43%	37%	7.42	2,968	36%	29%
P24	Ph-pdf-layout	f2d7b98	14,408	5.64	99	70%	58%	5.56	2,708	27%	18%
P25	Properties-maven-plugin	6cf7c2b	891	3.68	10	30%	22%	10.37	2,159	36%	32%
P26	Property-loader	928f414	1,860	2.75	105	85%	91%	3.49	3,421	74%	54%
P27	Restfb	35a34dd	42,022	20.99	1,273	59%	75%	34.21	7,100	68%	30%
P28	Sherdog-parser	aa6806a	1,546	24.10	12	68%	52%	182.16	1,212	73%	43%
P29	Trema-core	fa9f76d	3,285	3.51	60	72%	61%	3.99	4,332	44%	31%
P30	Velocity-config-tool	26226f5	358	4.68	15	100%	100%	24.12	18,927	24%	7%
P31	Visearch-sdk-java	0efcda3	7,643	3.90	151	75%	68%	134.82	3,496	73%	51%
Total	-	-	423,043	860.97	10,715	-	-	796.12	213,751	-	-
Avg	-	-	13,647	27.77	346	-	-	24.88	6895	-	-

`Round2Reducer` (i) searches for sets of values in the `Collector` that kill mutants that the first-round reduced set miss, and (ii) invokes `InlineTestConstructor` to create additional inline tests and add them to the set on which test-suite minimization is to be performed.

The mapping from the first-round reduced inline tests and the inline tests that are added back to mutants that they kill is fed into a test-suite minimization algorithm to obtain an intermediate second-round reduced set. We re-use a publicly-available implementation of four test-suite minimization algorithms [52]. This implementation was used in recent work [53–55], and consists of the greedy [58], GE and GRE [6], as well as HGS [20] algorithms.

Mutants may not be created for some target statements if mutation operators do not apply. We use all mutation operators in the Universal Mutation tool [17], which allows to easily specify line numbers to mutate. `Round2Reducer` adds inline tests generated for such target statement to the set obtained from test-suite minimization to obtain the final second-round reduced set.

5 EVALUATION

We answer the following research questions:

RQ1: How many inline tests does ExLi generate *before* reduction?

RQ2: How many inline tests does ExLi generate *after* reduction?

RQ3: How effective are the generated inline tests in terms of fault-detection capability, compared with unit tests?

RQ4: What are the runtime costs of ExLi?

We first use ExLi to discover target statements in open-source Java projects to obtain our evaluation subjects (Section 5.1). Then, we use ExLi to generate inline tests from developer-written and Randoop-generated unit tests (Section 5.2). Finally, we perform mutation analysis on the target statements to evaluate fault-detection capability of inline tests and unit tests (Section 5.3).

Experiment environment. We run all experiments on a machine with Intel Core i7-11700K @ 3.60GHz (8 cores, 16 threads) CPU, 64 GB RAM, Ubuntu 20.04, Java 8, and Maven 3.8.6.

5.1 Curating an Evaluation Dataset

We start with a large set of projects from a recent work on learning to complete unit tests [39]. Nie et al. [39] filter these projects based on their experimental requirements that are different from ours. So, we start from their original unfiltered set containing 1535 Java projects that use Maven, compile, and have appropriate licenses. To simplify our experiments, we select the subset of 1209 single-module projects. From these, we select the 128 actively-maintained projects that have commits after January 1, 2022. Next, we filter out projects in which developer-written unit tests fail (84 remain),

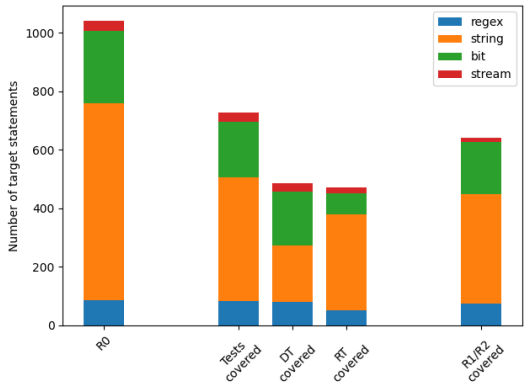

Figure 5: Number of target statements of each kind.

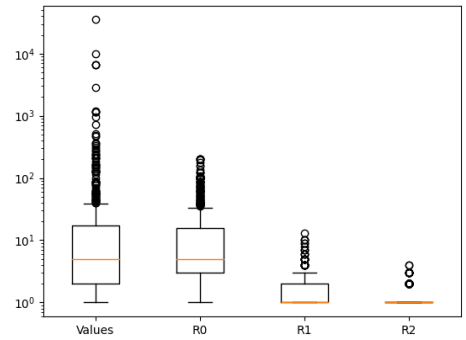

Figure 6: Number of inline tests per target statement.

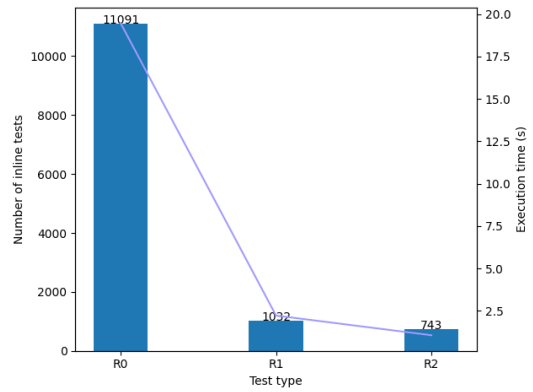

Figure 7: Total number of inline tests and execution time.

in which JaCoCo fails (73 remain), and in which Randoop or its generated unit tests fail (48 remain).

On these remaining 48 projects, we use ExLi to find target statements and generate inline tests. We filter out 6 projects that have no target statement of the kinds used by Liu et al. [29]; 1 project where all target statements are not covered by any unit test; and 1 project with target statements covered by unit tests, but for which ExLi does not generate any passing inline test. We further filter out projects where ExLi's instrumentation is incompatible (8 projects) or developer-written tests take more than one hour (1 project).

We use the remaining 31 projects as our evaluation subjects. Table 2 shows the names of these projects, the SHA and date of the commits that we use, total lines of Java code, and statistics about developer-written and Randoop-generated unit tests: test-running time, number of test methods, line coverage, and branch coverage.

Figure 5 shows statistics about the number of target statements in the 31 projects. ExLi initially finds 1041 target statements (35 for stream operations, 673 for string manipulation, 247 for bit manipulation, and 86 for regular expression). Of these, 727 target statements are covered by at least one unit test (486 are covered a developer-written unit test, and 471 are covered by a Randoop-generated unit test). After removing failing inline tests and discarding the corresponding target statements, ExLi generates inline tests for 642 target statements (15 for stream operations, 375 for string manipulation, 178 for bit manipulation, and 74 for regular expression). We use these 642 target statements in the rest of our evaluation.

5.2 Extracting Inline Tests

For each project in our dataset, we run Randoop with a time limit of 10 minutes, with other options set to their default values. Then, we run ExLi twice on each project: once with developer-written unit tests, and once with Randoop-generated unit tests. Note that the coverage information used by ExLi's `Round1Reducer` is persisted across the runs. We measure the size of four sets of variable values/inline tests at different stages of ExLi:

Values. The variable values collected in step ⑤ *in memory* without using the maximum generation limit of 200 inline tests per target statement. We collect Values for experimental reasons only; it is the upper limit of the unique number of inline tests that ExLi could generate. ExLi users do not have to collect Values.

R0. The inline tests generated by ExLi before reduction.

R1. The inline tests generated by ExLi after coverage-based first-round reduction (i.e., after step ⑦).

R2. The inline tests generated by ExLi after mutation-based second-round reduction (i.e., after step ⑨).

Figure 6 shows the distribution of the number of generated inline tests per target statement. The average number of inline tests per target statement for Values, R0, R1, and R2 are 139, 17, 2, and 1, respectively. The median number of inline tests per target statement for Values, R0, R1, and R2 are 5, 5, 1, and 1, respectively. The distribution of the number of inline tests per target statement for Values is long-tailed. It is to avoid running out of disk space for projects in this tail that we limit the number of inline tests generated per target statement to 200 (100 from developer-written unit tests and 100 from Randoop-generated tests). Observe that 95% of target statements are not affected by these limits. That is, the number of inline tests per target statement at 95% percentile is 170.

> **Answer to RQ1**. ExLi could generate an average of 139 inline tests per target statement. Limiting to at most 200 per target statement and removing failing ones, ExLi generates 17 inline tests per target statement on average.

Table 3: Mutants evaluation results.

PID	# Mutated target stmts	# Mutants	# inline tests		Mutation score				T(s)			
			R0	R1	R0(R2)	R1	Unit	Randoop	R0	R1	Unit	Randoop
P1	2	10	23	2	80.0	80.0	80.0	0.0	30.3	29.8	54.7	316.8
P2	1	1	16	2	100.0	100.0	100.0	100.0	3.6	3.5	4.6	9.0
P3	12	52	254	46	96.2	96.2	96.2	69.2	190.2	185.0	393.4	1237.2
P4	10	24	77	15	95.8	95.8	95.8	33.3	88.2	87.3	2109.8	548.4
P5	2	2	48	4	100.0	100.0	50.0	100.0	11.9	6.5	16.9	32.3
P6	3	32	84	11	68.8	62.5	53.1	0.0	117.8	102.0	140.8	367.5
P7	4	9	22	7	100.0	77.8	11.1	0.0	70.9	53.4	4675.3	467.0
P8	2	3	15	3	100.0	100.0	100.0	33.3	6.6	6.6	14.0	11.5
P9	14	56	94	26	82.1	82.1	100.0	3.6	501.8	500.0	31957.5	3902.1
P10	16	285	442	34	80.7	71.2	44.6	0.0	889.7	938.9	1431.0	3313.2
P11	9	63	146	13	93.7	92.1	39.7	49.2	178.8	179.8	367.2	601.5
P12	243	492	993	248	100.0	100.0	12.2	11.2	1712.2	1756.4	2129.6	4061.1
P13	2	25	21	2	92.0	44.0	28.0	0.0	60.8	59.4	84.6	272.2
P14	2	2	26	3	100.0	100.0	100.0	50.0	5.7	5.4	30.6	18.3
P15	3	5	214	6	100.0	100.0	60.0	80.0	30.0	17.2	156.5	37.0
P17	20	125	938	41	84.0	70.4	51.2	4.8	493.6	518.5	6345.8	3664.2
P18	1	5	1	1	40.0	40.0	40.0	0.0	13.7	13.7	21.5	51.1
P19	11	49	224	28	85.7	77.6	83.7	10.2	218.3	214.6	440.3	496.8
P20	30	474	1725	44	77.4	50.6	89.2	39.7	1207.3	1123.9	1753.3	4692.6
P21	38	132	915	64	100.0	92.4	90.2	72.7	857.2	619.4	1047.8	1971.6
P22	128	1420	2272	294	69.3	67.3	64.4	13.7	3751.2	3809.2	20329.8	6262.6
P23	5	25	36	14	92.0	92.0	92.0	8.0	69.5	69.0	271.8	222.7
P24	2	24	133	2	100.0	66.7	4.2	66.7	93.1	89.4	182.4	184.1
P25	4	12	54	4	33.3	0.0	25.0	0.0	68.8	67.8	92.3	157.8
P26	2	5	20	2	20.0	20.0	40.0	20.0	13.3	13.3	21.9	29.9
P27	16	70	707	29	58.6	58.6	97.1	100.0	621.5	609.2	1213.7	1444.5
P28	2	5	274	3	100.0	100.0	60.0	20.0	66.2	29.6	87.9	931.3
P29	4	10	25	4	90.0	60.0	100.0	30.0	23.5	23.5	48.9	70.3
P30	1	19	6	1	57.9	36.8	57.9	5.3	87.5	87.5	119.9	416.9
P31	4	12	4	4	100.0	100.0	33.3	100.0	28.0	27.7	56.1	1618.3
Total	593	3448	9809	957	2497.5	2234.0	1898.9	1020.9	11511.1	11247.6	75600.1	37409.7
Avg	19	111	316	31	80.6	72.1	61.3	32.9	371.3	362.8	2438.7	1206.8

Figure 7 shows the number of inline tests and their execution time (compilation time not included). The maximum possible number of inline tests without limits (Values) is 88682. After limiting the number of inline tests per target statement and removing failing ones, the resulting number of inline tests (R0) is 11091. Then, ExLi's coverage-based first-round reduction reduces the number of inline tests to 1032. During second-round reduction, ExLi first picks 842 inline tests that can kill mutants from R1, and then adds 1150 inline tests from R0 that can kill mutants missed by tests in R1. Applying test-suite minimization to the resulting 1992 inline tests yields 668 inline tests from the GHS algorithm. All four test-suite minimization algorithms (Greedy, GE, GRE, HGS) that we use produce the same number of inline tests, but the sets of inline tests that they produce are different. Finally ExLi adds the 75 inline tests from 49 target statements for which no *non-stillborn mutant* (i.e., a mutant that compiles) was created. So, the final second-round reduced set contains 743 inline tests.

The reduction ratio of first-round reduction is 90.7% compared to R0, and the reduction ratio of second-round reduction is 28.0% compared to R1. These reduction ratios shows that both rounds of ExLi's reduction are contribute to the overall reduction rate.

The total execution times of inline tests for R0, R1, and R2 (with HGS algorithm) are 19.46s, 2.18s, and 1.07s, respectively. That is, R1 reduced the execution time of R0 by 88.8%. Also, R2 reduced the execution time of R1 by 50.9% and execution time of R0 by 94.5%.

Answer to RQ2. ExLi's first-round and second-round reduction result in 90.7% and 93.3%, respectively, of all generated inline tests. That is, ExLi's two-round reduction produces 14.9*x* fewer inline tests.

5.3 Performing Mutation Analysis

Mutation testing is widely used to evaluate the quality of test suites [8, 45]. ExLi generates mutants as described under step ⑨ in Section 4. We report mutation analysis results based on the 593 target statements that have non-stillborn mutants. We also compare the mutation scores of inline tests generated by ExLi—at R0, R1, R2 stages—against unit tests.

Table 3 shows the mutation scores of initial inline tests (R0 & R2), round-one reduced inline tests (R1), developer-written unit tests, and Randoop-generated unit tests. Also, Figure 8 is a Venn diagram showing the overlap among the sets of mutants killed by R0 (same as R2), R1 reduced inline tests, developer-written unit tests and Randoop-generated unit tests.

The union of R0 inline tests, R1 inline tests, developer-written unit tests, and Randoop-generated unit tests kill 2924 mutants in

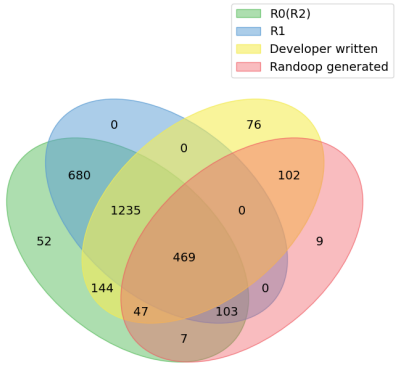

Figure 8: Sets of mutants killed by inline tests and unit tests.

total. Only 469 mutants are killed by the intersection of these sets of tests. Compared with R0 inline tests, R1 inline tests miss 250, or 8.5% of all killed mutants. Compared with developer-written unit tests and Randoop-generated unit tests, R0 inline tests miss 187 mutants, or 6.4% of all killed mutants. But, R0 kills 732 more mutants than both sets of unit tests, or 25.0% of all killed mutants.

We manually inspect surviving mutants that lead to loss of mutation scores when R1 is compared with R0. So far, we found two limitations of ExLi that lead to such losses: (1) There are multiple clauses in an `if` condition, but the mutation operator only modifies one of them. This limitation occurs because, unlike *pytest-inline*, I-Test does not yet support testing individual clauses in a condition. This limitation will go away as I-Test matures. (2) Multiple sets of values can kill a mutant but they all cover the target statement and its context in the same way as a chosen set of values that cannot kill the mutant. This is a limitation of coverage-based reduction as we discussed in Section 3.1.

Note that `Round2Reducer` recovers any losses in fault detection capability, regardless of the reason for the loss. So, the final second-round reduced set (R2), has the same mutation score as the initial set of inline tests without any reduction (R0). That is, ExLi's two-round reduction preserves fault-detection capability.

Observe from figure 8 that inline tests and unit tests are complementary in terms of their fault-detection capability. But, inline tests kill 732 more mutants than the combined sets of unit tests. That is, inline tests can enhance the fault-detection capability of the unit test suites from which they are extracted.

> **Answer to RQ3**. Inline tests generated by ExLi achieve an average mutation score of 80.6%, which is higher than the mutation scores of developer-written (61.3%) and Randoop-generated (32.9%) unit tests alone. A closer inspection of the sets of killed mutants shows that inline tests and unit tests have complementary fault-detection capabilities.

5.4 Measuring ExLi's Runtime Cost

Generating R0 and R1 unit tests took, on average across projects, 979.3s if one does not include the time to compile each mutant. The breakdown of the 979.3s average runtime of ExLi is as follows: 52.6s for running the unit tests, 555.4s for recording variable values, and 371.3s for running inline tests on mutants. Further, the average time to run inline tests on mutants is 0.11 seconds.

We omit compilation times, which is currently slow because we recompile the whole project for each mutant. Since generation is expected to be an offline process, we did not yet optimize the compilation portion of our ExLi implementation. In the future, we can make compilation significantly more efficient by compiling mutants in parallel, or by employing incremental compilation.

We are very encouraged by these early results on runtime costs, especially when compared with our estimated amount of time that it would take developers to write all 743 inline tests that ExLi produces after two rounds of reduction. In their user study, Liu et al. [29] found that participants spent around 360s to understand target statements and to write inline tests for Python. Assume that the times to understand target statements and write inline tests is uniformly distributed and are the same for Java and Python. Then, the participants in Liu et al.'s user study would have needed an average per project of 8628.4s to write these 743 inline tests.

> **Answer to RQ4**. Running ExLi takes 979.3s on average per project, when compilation times are not included. Our estimates, based on a prior user study, suggest that this average time is evidence that ExLi can reduce human effort to manually write inline tests. But more future work is needed to better achieve such savings in human effort.

6 DISCUSSION

Limitations. (1) ExLi uses coverage rate of the target statement and its context as feedback for its first round of reduction. If there are flaky tests [27, 33, 44], the coverage rate may fluctuate and render the choice of whether to use values for generating inline tests. We currently do not control for flaky tests in the unit tests that are being used for inline test generation. (2) Extracted inline tests may be flaky and fail if the expected output in the oracles that are generated depend on data that may change. Suppose that an inline test's oracle uses the current date or device configuration as its expected output. Then, when run on a different day or device, the inline test may fail. (3) When potential inputs to cause the target statement or its context to throw an exception, ExLi does not use such values to construct inline tests. The reason is that the I-Test [29] does not yet support the use of expected exceptions as test oracles. (4) We do not evaluate the extracted inline tests with developers of the open-source projects that we evaluate. So, it is not yet clear if developers would accept them. But, we have initial confidence from Liu et al.'s user study, which showed that participants find inline testing to be useful. As I-Test [29] becomes more mature, we will communicate more with developers who adopt it to see whether they want to use ExLi or accept the inline tests that we generate.

Threats to Validity. The code that we write to instrument target statements, collect coverage rates, and perform reduction could contain bugs. To mitigate this threat, at least two co-authors review the code, and multiple authors inspect the results. The findings in this paper could be limited to the projects that we evaluate and the unit tests that they contain. To mitigate this threat, we have used open-source projects with various characteristics and used Randoop

to generate more unit tests. The results that we report may not generalize to inline tests for other programming languages besides Java. We leave as future work to use *pytest-inline* [30] as a basis for extracting inline tests from Python unit tests. As support for inline testing grows for other languages, we believe that technique that we implement in ExLi is general enough to be instantiated.

Future Work. In other parts of this paper we discuss the need for future work to (1) support generation of inline tests for target statements in other types of program than the three that ExLi supports (`if` conditions, assignment statements, and declarations); (2) support other kinds of target statements other than the four that inline testing research (including this paper) has so far considered; (3) evaluate the use of other unit test generators other than Randoop; and (4) generate inline tests for other programming languages. In the future, we also plan to address the limitations of ExLi.

7 RELATED WORK

Single-Statement Bugs. Inline tests are partly motivated by recent work [24, 25, 28] showing that many bugs are caused by faults in single statements, and that unit tests miss such bugs. Liu et al. use inline tests to find single-statement bugs, and ExLi could help find more in the future. The ManySStuBs4J [25] dataset contains single-statement bugs that are curated by statically analyzing open-source Java projects and their version histories. As the ManySStuBs4J dataset evolves to capture more recent versions of those projects, it can become an alternate basis for evaluating the bug-detection capability of generated inline tests. We do not use the ManySStuBs4J dataset in this paper because (1) the filtering process that was followed to curate the dataset resulted in many false positives during our initial search for target statements; (2) the commits that are used in the dataset are from before 2019-01-28, so we had trouble running the unit tests in some of the projects.

Automatic Test Generation. Automatically generation of tests is a popular research topic and many test generation techniques have been proposed for Java [1, 3, 13, 15, 42, 51]. But, there was no technique before ExLi for automatically generating inline tests, which were recently proposed. Elbaum et al. [11] proposed a technique for extracting unit tests from system tests. ExLi is similar in spirit as it also extracts lower granularity tests from higher granularity tests. But ExLi differs in the particular lower and higher-granularity tests that it targets. Also, unlike Elbaum et al.'s technique, ExLi uses a novel coverage-based approach as well as traditional test-suite minimization to reduce the generated inline tests.

Test Suite Minimization/Reduction. Yoo and Harman [58] present a survey on test suite minimization. Zhang et al. [60] performed an empirical study on the effectiveness of test suite minimization techniques. Some widely used test suite minimization techniques include those that use (1) greedy algorithms [7], (2) heuristics [6, 20], and (3) integer linear programming [21]. We use a recent implementation of four of these algorithms [52] to further reduce the set of tests that ExLi generates after first-round reduction.

Shi et al. [53] compare the reduction rates and fault-detection capabilities of test suite minimization techniques. They find that techniques based on statement coverage reduce test-suite sizes by 62.9% but lose 20.5% in killed mutants. Conversely, techniques based on killed mutants achieve no loss in killed mutants but increase test-suite sizes by 10.9 percentage point, on average. Shi et al.'s study gives more confidence in preservation of fault-detection capability in ExLi's round two reduction, which is based on killed mutants.

Noemmer and Haas [40] recently compare test suite minimization techniques on open-source projects and find that, on average, test suites reduce by 70% while losing 12.5% of the fault-detection capability. Our results show that traditional test suite minimization reduces the round-one reduced inline tests by 28.0% and ExLi uses a feedback loop to preserve fault-detection capability.

Using Coverage as Feedback in Automated Testing. Coverage is a commonly used feedback for test generating [13, 14, 36] and test-suite reduction [19, 26, 35]. In our paper, the context-aware coverage is used as feedback for first round reduction. We are first to combine the change of coverage rate of target statement itself and the change of the block that contains the target statement as feedback.

Assertion/Invariants Generation. Program assertions/invariants are useful for checking the correctness of program states. Inline tests are similar to assert statements: both are co-located with program statements and they can be turned off in production. But inline tests are different: they allow to provide arbitrary inputs, expected outputs, and oracles for testing statements. Further, assert statements only run if they are situated in covered code, but inline tests will run in a different context even if the target statement is not covered by any test. Lastly, existing inline testing frameworks provide features that are typically not supported in assert statements: parameterized tests, repeating test runs (helpful to see if inline tests are flaky), grouping tests, and running tests in parallel.

There have been many techniques for automatically generating assertions and invariants, including those that (1) infer invariants from runtime information [5, 9, 12]; (2) generate assertions from comments and documentation [4, 16, 37]; and (3) learn assertions from code [10, 18, 39, 57, 59]. ExLi is most similar to approaches in the first category, as it extracts inline tests from runtime information. But, ExLi additionally (1) uses the collected information to construct inputs, expected outputs, and oracles for the generated inline tests; and (2) reduce the set of generated inline tests.

Mutation testing. Mutation testing is a technique for evaluating the effectiveness of a test suite [43, 46]. Popular mutation testing tools include PIT [56], MuJava [34], and universalmutator [17]. We choose to use universalmutator because it uses source code level replacement of operators and is easy to specify mutated line numbers.

8 CONCLUSION

In this paper, we presented ExLi, a technique for automatically generating inline tests with two rounds of reductions. The first round of reduction is based on context-aware coverage feedback, and the second round of reduction is based on killed mutants. We evaluate ExLi on 31 open-source Java projects and find that ExLi generates 11091 passing inline tests for 642 target statements. ExLi reduces initially generated inline tests by 14.9x without loss in mutation scores. ExLi enables developers to enhance the quality of their projects by easily obtaining inline tests.

REFERENCES

[1] Inc. Agitar. 2022. Agitar One. http://www.agitar.com/solutions/products/agitarone.html.

[2] M Moein Almasi, Hadi Hemmati, Gordon Fraser, Andrea Arcuri, and Janis Benefelds. 2017. An industrial evaluation of unit test generation: Finding real faults in a financial application. In *ICSE-SEIP*. 263–272.

[3] Andrea Arcuri and Gordon Fraser. 2016. Java Enterprise Edition Support in Search-Based JUnit Test Generation. In *International Symposium on Search Based Software Engineering*.

[4] Arianna Blasi, Alberto Goffi, Konstantin Kuznetsov, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Sergio Delgado Castellanos. 2018. Translating Code Comments to Procedure Specifications. In *ISSTA*. 242–253.

[5] Marat Boshernitsan, Roongko Doong, and Alberto Savoia. 2006. From Daikon to Agitator: lessons and challenges in building a commercial tool for developer testing. In *ISSTA*. 169–180.

[6] Tsong Yueh Chen and Man Fai Lau. 1970. Heuristics towards the optimization of the size of a test suite. *WIT Transactions on Information and Communication Technologies* 14 (1970).

[7] Tsong Yueh Chen and Man Fai Lau. 1998. A simulation study on some heuristics for test suite reduction. *Information and Software Technology* 40, 13 (1998), 777–787.

[8] Yiqun T Chen, Rahul Gopinath, Anita Tadakamalla, Michael D Ernst, Reid Holmes, Gordon Fraser, Paul Ammann, and René Just. 2020. Revisiting the relationship between fault detection, test adequacy criteria, and test set size. In *ASE*. 237–249.

[9] Christoph Csallner, Nikolai Tillmann, and Yannis Smaragdakis. 2008. DySy: Dynamic symbolic execution for invariant inference. In *ICSE*. 281–290.

[10] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. TOGA: a neural method for test oracle generation. In *ICSE*. 2130–2141.

[11] Sebastian Elbaum, Hui Nee Chin, Matthew B Dwyer, and Jonathan Dokulil. 2006. Carving differential unit test cases from system test cases. In *FSE*. 253–264.

[12] Michael D Ernst, Jeff H Perkins, Philip J Guo, Stephen McCamant, Carlos Pacheco, Matthew S Tschantz, and Chen Xiao. 2007. The Daikon system for dynamic detection of likely invariants. *SCP* 69, 1-3 (2007), 35–45.

[13] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *FSE*. 416–419.

[14] Gordon Fraser and Andrea Arcuri. 2012. Whole test suite generation. *IEEE Transactions on Software Engineering* 39, 2 (2012), 276–291.

[15] Indradeep Ghosh, Nastaran Shafiei, Guodong Li, and Wei-Fan Chiang. 2013. JST: An automatic test generation tool for industrial Java applications with strings. *2013 35th International Conference on Software Engineering (ICSE)* (2013), 992–1001.

[16] Alberto Goffi, Alessandra Gorla, Michael D. Ernst, and Mauro Pezzè. 2016. Automatic Generation of Oracles for Exceptional Behaviors. In *ISSTA*. 213–224.

[17] Alex Groce, Josie Holmes, Darko Marinov, August Shi, and Lingming Zhang. 2018. An extensible, regular-expression-based tool for multi-language mutant generation. In *ICSE-Demo*. 25–28.

[18] Long H. Pham, Ly Ly Tran Thi, and Jun Sun. 2017. Assertion generation through active learning. In *ICFEM*. Springer, 174–191.

[19] Preethi Harris and Raju Nedunchezhian. 2015. A greedy approach for coverage-based test suite reduction. *Int. Arab J. Inf. Technol.* 12 (2015), 17–23.

[20] M Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. 1993. A methodology for controlling the size of a test suite. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 2, 3 (1993), 270–285.

[21] Joshua Hartmann and Dave J Robson. 1989. Revalidation during the software maintenance phase. In *Proceedings. Conference on Software Maintenance-1989*. IEEE, 70–80.

[22] Dennis Jeffrey and Neelam Gupta. 2007. Improving fault detection capability by selectively retaining test cases during test suite reduction. *IEEE Transactions on software Engineering* 33, 2 (2007), 108–123.

[23] jsonSchemaValidator 2022. JSON Schema Validator. https://github.com/networknt/json-schema-validator.

[24] Arthur V Kamienski, Luisa Palechor, Cor-Paul Bezemer, and Abram Hindle. 2021. PySStuBs: Characterizing single-statement bugs in popular open-source Python projects. In *MSR*. 520–524.

[25] Rafael-Michael Karampatsis and Charles Sutton. 2020. How often do single-statement bugs occur? The ManySStuBs4J dataset. In *MSR*. 573–577.

[26] Saif Ur Rehman Khan, Sai Peck Lee, Reza Meimandi Parizi, and Manzoor Elahi. 2014. A code coverage-based test suite reduction and prioritization framework. *2014 4th World Congress on Information and Communication Technologies (WICT 2014)* (2014), 229–234.

[27] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)* (2019), 312–322.

[28] Jasmine Latendresse, Rabe Abdalkareem, Diego Elias Costa, and Emad Shihab. 2021. How effective is continuous integration in indicating single-statement bugs?. In *MSR*. 500–504.

[29] Yu Liu, Pengyu Nie, Owolabi Legunsen, and Milos Gligoric. 2022. Inline Tests. In *ASE*.

[30] Yu Liu, Zachary Thurston, Alan Han, Pengyu Nie, Milos Gligoric, and Owolabi Legunsen. 2023. pytest-inline: An Inline Testing Tool for Python. In *ICSE-Demo*. to appear.

[31] LogstashGelf 2022. Mp911de Logstash Gelf. https://github.com/mp911de/logstash-gelf.git.

[32] LogstashGelf 2022. XStream developer. https://x-stream.github.io/index.html.

[33] Qingzhou Luo, Lamyaa Eloussi, Farah Hariri, and Darko Marinov. 2014. Can We Trust Test Outcomes ?

[34] Yu-Seung Ma, A. Jefferson Offutt, and Yong Rae Kwon. 2005. MuJava: an automated class mutation system. *Software Testing* 15 (2005).

[35] Alessandro Marchetto, Giuseppe Scanniello, and Angelo Susi. 2019. Combining Code and Requirements Coverage with Execution Cost for Test Suite Reduction. *IEEE Transactions on Software Engineering* 45 (2019), 363–390.

[36] Phil McMinn. 2004. Search-based software test data generation: a survey. *Software testing, Verification and reliability* 14, 2 (2004), 105–156.

[37] Manish Motwani and Yuriy Brun. 2019. Automatically Generating Precise Oracles from Structured Natural Language Specifications. In *ICSE*. 188–199.

[38] Mountainminds GmbH & Co. KG and Contributors. 2022. JaCoCo - Java Code Coverage Library. https://www.jacoco.org/jacoco.

[39] Pengyu Nie, Rahul Banerjee, Junyi Jessy Li, Raymond J. Mooney, and Milos Gligoric. 2023. Learning deep semantics for test completion. In *ICSE*. 1–12.

[40] Raphael Noemmer and Roman Haas. 2019. An evaluation of test suite minimization techniques. In *Software Quality: Quality Intelligence in Software and Systems Engineering: 12th International Conference, SWQD 2020, Vienna, Austria, January 14–17, 2020, Proceedings*. Springer, 51–66.

[41] Oracle. 2022. Chapter 4. The class file format. https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-4.html#jvms-4.7.3.

[42] Carlos Pacheco and Michael D Ernst. 2007. Randoop: feedback-directed random testing for Java. In *OOPSLA*. 815–816.

[43] Mike Papadakis, Marinos Kintis, Jie M. Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Chapter Six - Mutation Testing Advances: An Analysis and Survey. *Adv. Comput.* 112 (2019), 275–378.

[44] Owain Parry, Gregory M. Kapfhammer, Michael C Hilton, and Phil McMinn. 2022. A Survey of Flaky Tests. *ACM Trans. Softw. Eng. Methodol.* 31 (2022), 17:1–17:74.

[45] Goran Petrović, Marko Ivanković, Gordon Fraser, and René Just. 2021. Practical mutation testing at scale: A view from Google. *TSE* 48, 10 (2021), 3900–3912.

[46] Goran R. Petrovi'c, Marko Ivankovi'c, Gordon Fraser, and René Just. 2021. Does Mutation Testing Improve Testing Practices? *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)* (2021), 910–921.

[47] PytestInline 2023. Inline Testing Team. https://github.com/pytest-dev/pytest-inline.

[48] PytestInlinePyPi 2023. Inline Testing Team. https://pypi.org/project/pytest-inline.

[49] Brian Robinson, Michael D Ernst, Jeff H Perkins, Vinay Augustine, and Nuo Li. 2011. Scaling up automated test generation: Automatically generating maintainable regression unit tests for programs. In *ASE*. IEEE, 23–32.

[50] Gregg Rothermel, Mary Jean Harrold, Jeffery Ostrin, and Christie Hong. 1998. An empirical study of the effects of minimization on the fault detection capabilities of test suites. In *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*. IEEE, 34–43.

[51] Sina Shamshiri, René Just, José Miguel Rojas, Gordon Fraser, Phil McMinn, and Andrea Arcuri. 2015. Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges (t). In *ASE*. IEEE, 201–211.

[52] August Shi. 2023. Collection of scripts to conduct test-suite reduction. https://github.com/august782/testsuite-reduction.

[53] August Shi, Alex Gyori, Milos Gligoric, Andrey Zaytsev, and Darko Marinov. 2014. Balancing trade-offs in test-suite reduction. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*. 246–256.

[54] August Shi, Alex Gyori, Suleman Mahmood, Peiyuan Zhao, and Darko Marinov. 2018. Evaluating test-suite reduction in real software evolution. In *ISSTA*. 84–94.

[55] August Shi, Tifany Yung, Alex Gyori, and Darko Marinov. 2015. Comparing and combining test-suite reduction and regression test selection. In *FSE*. 237–247.

[56] Pitest Team. 2022. PIT - Mutation Testing for Java. https://pitest.org/.

[57] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *ICSE*. 1398–1409.

[58] Shin Yoo and Mark Harman. 2012. Regression testing minimization, selection and prioritization: a survey. *STVR* 22, 2 (2012), 67–120.

[59] Hao Yu, Yiling Lou, Ke Sun, Dezhi Ran, Tao Xie, Dan Hao, Ying Li, Ge Li, and Qianxiang Wang. 2022. Automated assertion generation via information retrieval and its integration with deep learning. In *ICSE*. 163–174.

[60] Lingming Zhang, Darko Marinov, Lu Zhang, and Sarfraz Khurshid. 2011. An empirical study of junit test-suite reduction. In *ISSRE*. IEEE, 170–179.